Branching AI Conversation Workspace

Complete Technical Implementation Guide

Parallel Development Playbook for a 4-Developer Team

Version 1.0 | April 2026 | Confidential

Table of Contents

(Generate after opening in Word: References > Table of Contents)

1. Product Overview and Architecture Philosophy

This document is the single source of truth for building the Branching AI Conversation Workspace from scratch. It covers every module, every interface contract, and how four developers work in parallel without blocking each other. The core product lets users chat with multiple LLMs, fork conversations into independent branches from any message, visually navigate a conversation tree, and manage context budgets automatically.

The architecture follows three non-negotiable principles:

- **Immutable messages:** Every message is an atomic, content-addressable node. Once created, nodes are never mutated. This enables deterministic checkpointing and branch isolation.
- **Adjacency-list tree:** The conversation graph is stored as a self-referential parent_id adjacency list in PostgreSQL, queried via recursive CTEs, with optional ltree optimization for deep trees.
- **Provider-agnostic LLM layer:** The Vercel AI SDK abstracts all model providers behind a unified streaming interface. Adding a new model is a single-line registry entry.

1.1 High-Level System Architecture

The system is composed of seven layers, each owned by a specific developer but designed with clear API contracts so all four developers work simultaneously from Day 1.

Layer	Components	Primary Owner	Secondary
Client Layer	React 18 + React Flow + Zustand	Dev A (Frontend)	Dev C
API Layer	Fastify + Zod validation + SSE streaming	Dev B (Backend)	Dev C
Data Layer	PostgreSQL (Drizzle ORM) + Redis cache	Dev B (Backend)	Dev D
LLM Services	Vercel AI SDK + provider adapters + context assembler	Dev C (Full-stack/LLM)	Dev B
Auth Layer	Clerk (frontend + backend middleware)	Dev A + Dev C	—
Real-time Layer	WebSocket server + Yjs CRDT (future)	Dev C	Dev A
Infra / CI-CD	AWS ECS Fargate + RDS + ElastiCache + GitHub Actions	Dev D (DevOps/Product)	—

2. Team Structure and Parallel Work Strategy

All four developers write code every day. There are zero pure managers. Parallelism is achieved through strict API contract definitions agreed on before implementation begins. Each developer owns a vertical domain but pairs across boundaries for integration.

2.1 Developer Roles

Developer	Title	Primary Domain	Secondary
Dev A	Frontend Developer	React 18, React Flow, Zustand, UI/UX, landing page	Export UI, accessibility
Dev B	Backend Developer	Fastify API, PostgreSQL, Redis, Drizzle ORM, migrations	API design, rate limiting, caching
Dev C	Full-stack / LLM Engineer	LLM adapter layer, streaming, context budget, prompt engineering	Backend flex, auth middleware, merge logic
Dev D	DevOps / Product Lead	AWS infra, CI/CD, monitoring, QA, scope enforcement	Landing page deploy, load testing, launch prep

2.2 Parallelism Rules

- **Contract-first development:** Before any implementation begins, Dev A and Dev B agree on an OpenAPI spec for every endpoint. Dev A builds against mock data while Dev B implements the real API. This eliminates frontend-blocks-on-backend delays.
- **Monorepo structure:** A single GitHub repository with /apps/web (frontend), /apps/api (backend), /packages/shared (types, contracts, utilities). All developers commit to the same repo.
- **Feature branches:** Each developer works on feature branches. PRs require one approval from any other developer. Dev D owns the CI/CD pipeline and enforces green builds before merge.
- **Daily async standup:** 10-minute Slack post at 9:30am. Friday 30-minute demo + retro. Architectural decisions logged in shared Notion within 1 hour.

2.3 Integration Points and Dependency Resolution

The following matrix shows which modules depend on which, and how to unblock parallel work:

Dependency	Blocked Dev	Unblock Strategy
Chat UI needs Message API	Dev A	Dev A uses mock API server with agreed JSON contracts. Dev B delivers real API independently.
Streaming UI needs SSE endpoint	Dev A	Dev C builds SSE endpoint in parallel. Dev A tests with a local mock SSE stream.
Branch UI needs Branch API	Dev A	Dev A builds branch context menu and modal against mock endpoints.
Context assembly needs DB schema	Dev C	Dev B delivers schema first. Dev C writes context assembly logic against seed data.

LLM streaming needs auth middleware	Dev C	Dev C implements Clerk middleware on backend while Dev A integrates Clerk on frontend in parallel.
Landing page needs screenshots	Dev D	Dev D wireframes early, adds real screenshots once UI stabilizes.

3. Module 1: Database Schema and Data Layer

Owner: Dev B | **Depends on:** Nothing (start immediately) | **Blocks:** Dev C (context assembly), Dev A (API contracts)

3.1 Technology Stack

- PostgreSQL 15+ on AWS RDS (db.t4g.micro for development, db.r6g.large for production)
- Drizzle ORM for type-safe schema definition and migrations
- ltree extension enabled from Day 1 (even if not used initially)
- Redis 7 on ElastiCache (cache.t4g.micro) for caching and rate limiting

3.2 Core Tables

The schema uses a normalized relational model with nine core tables. The most critical design decision is the messages table, which uses a self-referential parent_id to form the conversation DAG (directed acyclic graph).

3.2.1 users

Managed by Clerk. This table mirrors Clerk user data for foreign key references.

Column	Type	Constraints	Notes
id	VARCHAR(255)	PK	Clerk user ID
email	VARCHAR(255)	UNIQUE, NOT NULL	From Clerk
display_name	VARCHAR(255)	NOT NULL	
avatar_url	TEXT	NULLABLE	
created_at	TIMESTAMPTZ	DEFAULT NOW()	

3.2.2 workspaces

Top-level container for conversation trees. Each user can have multiple workspaces.

Column	Type	Constraints	Notes
id	UUID	PK, DEFAULT gen_random_uuid()	
owner_id	VARCHAR(255)	FK → users.id, NOT NULL	
name	VARCHAR(255)	NOT NULL	User-provided name
system_prompt	TEXT	NULLABLE	Default system prompt for all conversations
created_at	TIMESTAMPTZ	DEFAULT NOW()	
updated_at	TIMESTAMPTZ	DEFAULT NOW()	

3.2.3 messages (Critical — the conversation DAG)

Every single message is an atomic, immutable row. The parent_id column creates the tree structure. The content_hash enables content-addressable checkpointing.

Column	Type	Constraints	Notes
id	UUID	PK, DEFAULT gen_random_uuid()	
workspace_id	UUID	FK → workspaces.id, NOT NULL	
parent_id	UUID	FK → messages.id, NULLABLE	NULL = root message
branch_id	UUID	FK → branches.id, NOT NULL	Which branch this belongs to
author_id	VARCHAR(255)	FK → users.id, NOT NULL	
role	VARCHAR(20)	NOT NULL, CHECK IN (user, assistant, system)	
content	TEXT	NOT NULL	Message body (markdown)
model	VARCHAR(100)	NULLABLE	e.g. gpt-4o, claude-sonnet-4-20250514
token_count	INTEGER	NOT NULL, DEFAULT 0	Pre-computed at creation
content_hash	VARCHAR(64)	NOT NULL, UNIQUE	SHA-256(role + content + parent_id)
created_at	TIMESTAMPTZ	DEFAULT NOW()	

3.2.4 branches

Column	Type	Constraints	Notes
id	UUID	PK	
workspace_id	UUID	FK → workspaces.id	
name	VARCHAR(255)	NOT NULL	User-given branch name
fork_message_id	UUID	FK → messages.id, NULLABLE	NULL = root/main branch
model	VARCHAR(100)	NOT NULL, DEFAULT gpt-4o	Active model for this branch
is_active	BOOLEAN	DEFAULT false	Currently active

			branch in workspace
head_message_id	UUID	FK → messages.id, NULLABLE	Latest message in branch
created_at	TIMESTAMPTZ	DEFAULT NOW()	

3.2.5 Additional Tables

The following tables complete the schema:

- **llm_providers:** Registry of available LLM providers with id, name, api_endpoint, model_id, context_window, is_active.
- **snapshots:** Immutable checkpoints with snapshot_hash (SHA-256 of ordered content hashes), branch_id, message_id, created_at.
- **merge_records:** Tracks merge operations with source_branch_id, target_branch_id, strategy (concat/summarize), injected_content, created_at.
- **context_summaries:** Cached progressive summaries keyed by first_node_hash:last_node_hash, content, token_count, created_at.

3.3 Critical Indexes

These indexes are essential for performance. Deploy them in the initial migration:

- messages(workspace_id, parent_id) — tree traversal
- messages(branch_id, created_at) — branch message listing
- messages(content_hash) — content-addressable lookup and deduplication
- branches(workspace_id) — list branches per workspace
- branches(fork_message_id) — find branches forked from a message

3.4 The Ancestor Chain Query (Recursive CTE)

This is the most critical query in the entire system. It assembles the linear context for any message by walking up the tree to the root. Dev C uses this to build the LLM context payload.

```
WITH RECURSIVE ancestors AS ( SELECT id, parent_id, role, content, token_count, 1 AS depth FROM messages WHERE id = $leafNodeId UNION ALL SELECT m.id, m.parent_id, m.role, m.content, m.token_count, a.depth + 1 FROM messages m JOIN ancestors a ON m.id = a.parent_id ) SELECT * FROM ancestors ORDER BY depth DESC;
```

3.5 Redis Cache Strategy

Dev B implements a cache-aside pattern for workspace trees and branch metadata:

- **Key:** tree:{workspaceId} — full tree structure JSON, TTL 300s
- **Key:** branch:{branchId}:tokens — total token count, TTL 60s
- **Key:** summary:{firstHash}:{lastHash} — cached context summaries, TTL 3600s
- **Invalidation:** Any mutation to messages or branches invalidates the relevant tree cache.
- **Rate limiting:** Sliding window with sorted sets: ZADD ratelimit:{userId} {timestamp} {requestId}, 20 requests/minute for LLM endpoints.

4. Module 2: API Layer

Owner: Dev B | **Depends on:** Database schema (Module 1, same owner) | **Blocks:** Dev A (needs API contracts on Day 1)

4.1 Technology Stack

- Fastify 4 with TypeScript
- Zod for request/response validation
- Pino for structured logging
- Helmet + CORS for security
- Clerk Fastify plugin for JWT auth middleware

4.2 API Endpoint Reference

Dev B publishes this contract on Day 1. Dev A builds against these interfaces immediately using mock data.

4.2.1 Workspace Endpoints

Method	Path	Request Body	Response	Notes
POST	/api/workspaces	{ name, systemPrompt? }	Workspace object	Creates workspace + default main branch
GET	/api/workspaces	—	Workspace[]	List user workspaces
GET	/api/workspaces/:id	—	Workspace + branches	Full workspace with branch list
GET	/api/workspaces/:id/tree	—	Full message tree	All nodes across all branches
PATCH	/api/workspaces/:id	{ name?, systemPrompt? }	Updated workspace	
DELETE	/api/workspaces/:id	—	204	Soft delete

4.2.2 Message Endpoints

Method	Path	Request Body	Response	Notes
POST	/api/messages	{ workspaceId, branchId, parentId, content }	Message object	Creates user message node

GET	/api/messages/:id/ancestors	—	Message[] (ordered root→leaf)	Ancestor chain via recursive CTE

4.2.3 Branch Endpoints

Method	Path	Request Body	Response	Notes
POST	/api/branches/fork	{ workspaceId, forkMessageId, name, model? }	Branch object	Create branch from message
GET	/api/branches/:id/messages	—	Message[] (linear path)	Messages in branch order
PATCH	/api/branches/:id	{ name?, model? }	Updated branch	Rename or change model
POST	/api/branches/:id/activate	—	200	Set as active branch
DELETE	/api/branches/:id	—	204	Archive (soft delete)
GET	/api/branches/:id/token-count	—	{ used, available, percentage }	Context budget info

4.2.4 Streaming Endpoint (Owned by Dev C)

Method	Path	Request Body	Response	Notes
POST	/api/chat/stream	{ branchId, model }	SSE stream (text/event-stream)	Dev C implements; uses AI SDK streamText()

4.2.5 Export Endpoints

Method	Path	Query Params	Response	Notes
GET	/api/export/branch/:id	format=json|markdown	File download	Linearized branch export
GET	/api/export/workspace/:id	format=json	File download	Full tree export

4.3 Error Response Format

All errors follow a consistent shape. Dev A relies on this for UI error handling:

```
{ "error": { "code": "BRANCH_NOT_FOUND", "message": "Branch with ID xyz does not exist",
"statusCode": 404 } }
```

5. Module 3: LLM Service Layer

Owner: Dev C | **Depends on:** Database schema (Dev B delivers first), Auth middleware (Dev C self-implements) | **Blocks:** Dev A (streaming hook)

5.1 Technology Stack

- Vercel AI SDK (ai + @ai-sdk/openai + @ai-sdk/anthropic)
- gpt-tokenizer for OpenAI token counting
- Model registry pattern for provider abstraction

5.2 Model Registry

A central registry maps model identifiers to their SDK providers, context window sizes, and tokenizer configs. Adding a new model is a single object entry.

Model ID	Provider	Context Window	Tokenizer	Priority
gpt-4o	OpenAI	128,000 tokens	o200k_base (gpt-tokenizer)	Launch model
claude-sonnet-4-20250514	Anthropic	200,000 tokens	Anthropic tokenizer	Second model
claude-3.5-haiku	Anthropic	200,000 tokens	Anthropic tokenizer	Fast/cheap option

5.3 Context Assembly Algorithm

This is the core intellectual property of the product. When a user sends a message in a branch, the context assembler builds the exact right payload for the LLM:

- **Step 1:** Retrieve the ancestor chain from the current message up to the root using the recursive CTE (Dev B provides this query).
- **Step 2:** Compute total token count by summing pre-computed token_count values from all ancestor nodes.
- **Step 3:** If total tokens < 80% of model context window, send the full ancestor chain as-is.
- **Step 4:** If total tokens >= 80%, trigger progressive summarization: keep the last 10 messages verbatim, summarize all older messages using the same LLM with prompt: "Condense the following conversation history into key facts and decisions, preserving all actionable information."
- **Step 5:** Cache the summary in Redis keyed by summary:{firstNodeHash}:{lastNodeHash} with 1-hour TTL.
- **Step 6:** Assemble final payload: [system_prompt, cached_summary (if any), recent_messages] and send to LLM.

5.4 Streaming Implementation

Dev C builds the SSE streaming endpoint using the AI SDK:

- **Endpoint:** POST /api/chat/stream accepts { branchId, model }.
- **Flow:** Assemble context → call streamText() with provider → pipe toDataStream() to response → on stream complete, persist the full assistant response as a new message node.

- **Client disconnect:** Listen to request.socket.on('close') to abort the LLM stream if the user navigates away.
- **Error handling:** Retry with exponential backoff (3 attempts). If primary model fails, offer fallback model. 30-second timeout per model.

5.5 Content-Addressable Hashing

Every message node gets a content_hash = SHA-256(role + content + parent_id). This is computed at node creation time and stored in the database. The hash enables:

- Deduplication: ON CONFLICT (content_hash) DO NOTHING prevents duplicate nodes.
- Deterministic checkpoints: A snapshot is the SHA-256 of ordered content hashes in a branch.
- Cache keying: Context summaries are cached by the hash range they cover.

5.6 Token Counting

Dev C pre-computes token counts at message creation time using gpt-tokenizer (for OpenAI models) and stores the count in the messages.token_count column. For Anthropic models, a separate tokenizer is used. The token count API endpoint (GET /api/branches/:id/token-count) sums ancestor chain tokens and compares against the model context window.

6. Module 4: Frontend Application

Owner: Dev A | **Depends on:** API contracts (published Day 1 by Dev B), SSE endpoint (Dev C) | **Blocks:** Nothing (uses mocks until real APIs are ready)

6.1 Technology Stack

- React 18 + TypeScript + Vite
- React Flow for canvas/tree visualization
- @dagrejs/dagre for automatic tree layout (rankdir: TB)
- Zustand for state management
- react-markdown for message rendering
- @clerk/clerk-react for authentication
- Tailwind CSS for styling

6.2 Application Architecture

The frontend is structured into five major UI regions:

Region	Component	Description
Header Bar	AppHeader	Logo, workspace selector, user menu (Clerk UserButton), settings
Left Sidebar	BranchSidebar	Collapsible list of all branches with fork point, message count, last activity. Click to navigate.
Main Canvas	ConversationCanvas	React Flow canvas showing the full message tree with dagre auto-layout
Linear Chat Panel	ChatPanel	Focused linear view of the active branch. Toggleable alongside the canvas (dual-view).
Chat Input	ChatInput	Text area + submit button + model selector dropdown + context budget indicator

6.3 React Flow Integration

6.3.1 Custom Message Node

Each message renders as a custom React Flow node showing: role badge (User/Assistant), model badge (GPT-4o/Claude), truncated content preview, timestamp, and a context menu trigger.

6.3.2 Dagre Layout Algorithm

The tree is laid out using dagre with direction top-to-bottom. Because message nodes have variable heights, Dev A must implement a measure-then-layout pattern:

- **Phase 1:** Render all nodes at position {0, 0} in a hidden container to measure actual DOM dimensions.
- **Phase 2:** Pass measured dimensions to dagre, compute layout positions.
- **Phase 3:** Apply computed positions to React Flow nodes with animation.

6.3.3 Branch Visualization

When a user forks from a message, the new branch appears as a diverging path. The active branch path is highlighted with colored edges and fully opaque nodes. Inactive branches are dimmed (reduced opacity). The MiniMap component provides an overview for large trees.

6.4 Streaming Integration

Dev A builds a useStreamingChat hook that:

- Uses fetch() with ReadableStream reader to consume the SSE stream from Dev C's endpoint.
- Buffers incoming tokens using a ref and flushes to Zustand store via requestAnimationFrame batching to prevent render thrashing.
- Only re-renders the active streaming node using Zustand selective subscriptions.
- On stream completion, fetches the persisted message from the API to get the final node with content_hash and token_count.

6.5 Branch Operations UI

Right-clicking any message node opens a context menu (onNodeContextMenu) with options:

- "Branch from here" → opens modal with branch name input + model selector → calls POST /api/branches/fork
- "Copy message" → copies content to clipboard
- "View ancestors" → highlights the ancestor chain on the canvas

6.6 Context Budget Indicator

A progress bar below the chat input shows token usage: green (<60%), yellow (60–80%), red (>80%). Tooltip shows breakdown: recent messages tokens, summarized history tokens, available tokens. Data comes from GET /api/branches/:id/token-count (polled on branch switch and after each message).

6.7 Export UI

Export modal: "Export branch as..." with JSON and Markdown options. "Export workspace" for full tree. Triggers client-side file download via the export API endpoints.

7. Module 5: Authentication and Authorization

Owners: Dev A (frontend) + Dev C (backend middleware) | **Depends on:** Nothing (start immediately) | **Blocks:** All API calls

7.1 Clerk Integration

Clerk handles all authentication complexity: sign-up, sign-in, session management, JWT issuance.

Frontend (Dev A)

- Wrap app root in <ClerkProvider publishableKey={...}>
- Build sign-in/sign-up pages using Clerk pre-built components
- Add <UserButton /> to header for profile/sign-out
- Protect main routes with Clerk's <SignedIn> / <SignedOut> guards

Backend (Dev C)

- Install @clerk/fastify plugin
- Build auth middleware that extracts userId from Clerk JWT on every request
- Attach userId to request context for use in all downstream handlers
- All workspace/message/branch endpoints verify ownership: the authenticated user must own the workspace being accessed

7.2 Authorization Rules

- Users can only access their own workspaces (workspace.owner_id === request.userId)
- All message and branch operations validate workspace ownership transitively
- Admin endpoints (GET /admin/usage) are restricted to designated admin user IDs configured via environment variable

8. Module 6: Infrastructure and CI/CD

Owner: Dev D | **Depends on:** Nothing (start immediately) | **Blocks:** All developers (need a deployable environment)

8.1 AWS Architecture

Service	Configuration	Purpose
ECS Fargate	Express Mode, min 1 / max 4 tasks, CPU target 70%	Backend container hosting with auto-scaling
RDS PostgreSQL	db.t4g.micro (dev), db.r6g.large (prod), automated backups	Primary database
ElastiCache Redis	cache.t4g.micro	Caching, rate limiting, session data
ECR	Repository per service	Docker image registry
ALB	Auto-provisioned by ECS Express Mode	Load balancer with HTTPS termination
ACM	SSL certificate for custom domain	HTTPS
Route 53	Alias record → ALB	DNS
SSM Parameter Store	API keys, secrets	Secret management (no hardcoded keys)
CloudWatch	Logs (30-day retention) + alarms	Monitoring: CPU >80%, 5xx >5%, memory >85%

8.2 CI/CD Pipeline (GitHub Actions)

Dev D sets this up on Day 1 so every developer can deploy from the first commit:

- **Trigger:** Push to main branch.
- **Steps:** Lint + type-check → run tests → build Docker image → push to ECR (tagged with commit SHA) → deploy to ECS staging.
- **Auth:** OIDC federation for AWS credentials (no static access keys in GitHub).
- **Environments:** staging (auto-deploy from main) and production (manual promotion).

8.3 Monitoring and Observability

- Sentry for error tracking (frontend @sentry/react + backend @sentry/node)
- CloudWatch Dashboard with key metrics: request latency, error rate, active connections, LLM API latency
- CloudWatch Alarms: CPU >80%, 5xx rate >5%, memory >85%
- Structured JSON logging via Pino with request IDs for tracing

8.4 Environment Configuration

Variable	Source	Used By

DATABASE_URL	SSM Parameter Store	Backend
REDIS_URL	SSM Parameter Store	Backend
CLERK_SECRET_KEY	SSM Parameter Store	Backend
CLERK_PUBLISHABLE_KEY	Build-time env	Frontend
OPENAI_API_KEY	SSM Parameter Store	Backend (LLM layer)
ANTHROPIC_API_KEY	SSM Parameter Store	Backend (LLM layer)
SENTRY_DSN	Build-time env	Frontend + Backend

8.5 Estimated Monthly Infrastructure Cost

Service	Configuration	Est. Monthly Cost
ECS Fargate	1 task (0.5 vCPU, 1GB RAM)	$15–30
RDS PostgreSQL	db.t4g.micro	$15–25
ElastiCache Redis	cache.t4g.micro	$12–18
ALB	Low traffic	$18–25
CloudWatch + ECR + Route 53	Standard usage	$5–10
		$65–130 total

9. Module 7: Branching Engine (Core Product Logic)

Owners: Dev B (API), Dev C (context isolation), Dev A (UI) | All three work in parallel on their respective layers.

9.1 Branch Creation Flow

When a user right-clicks a message and selects "Branch from here":

- **Frontend (Dev A):** Opens modal with branch name + model selector. On submit, calls POST /api/branches/fork with { workspaceId, forkMessageId, name, model }.
- **Backend (Dev B):** Creates a new row in branches with fork_message_id pointing to the selected message. Returns the new branch object. Invalidates tree cache.
- **Context Isolation (Dev C):** When a message is sent in the new branch, the context assembler walks from the new branch head up through the fork point to the root. Only messages in this linear path are included. Messages from sibling branches are structurally excluded because they have different parent_id chains.

9.2 Branch Isolation Guarantee

Branch isolation is the most critical correctness property in the system. It is guaranteed by the data model itself: the recursive CTE ancestor chain query only follows parent_id links. Since messages in Branch A have parent_ids that trace through Branch A's fork point, they never include messages from Branch B. Dev D must verify this with integration tests covering:

- Fork from root, send messages in both branches, verify zero cross-contamination
- Fork from a fork point (nested branches), verify each branch has independent context
- Three or more branches from the same message, verify complete isolation

9.3 Branch Navigation

Switching branches does not copy or move data. It simply changes which branch is marked active (branches.is_active) and pans the React Flow canvas to that branch's head node. The context assembler always works from the active branch's head message.

9.4 Merge (Concat Strategy — v1 Only)

If implementation capacity allows, Dev C implements a basic concat merge: selected messages from a source branch are injected as a context summary node in the target branch. A merge_records entry tracks the operation. Advanced strategies (summarize, diff) are deferred to v2.

10. Module 8: Context Budget Management

Owner: Dev C | **Depends on:** Token counting (Module 3), ancestor chain query (Module 1) | **Blocks:** Nothing (enhancement layer)

10.1 Why Context Budgets Matter

Without context budget management, deep branches silently exceed the model's context window, causing either API errors or degraded response quality. The budget system makes this invisible problem visible and automatically manages it.

10.2 Progressive Summarization Algorithm

When the total tokens in an ancestor chain approach the model limit:

- **Threshold:** 80% of model context window (e.g., 102,400 tokens for GPT-4o).
- **Action:** Keep the most recent 10 messages verbatim. Summarize all older messages using the same LLM.
- **Prompt:** "Condense the following conversation history into key facts and decisions, preserving all actionable information."
- **Caching:** Summary is cached in Redis keyed by summary:{firstNodeHash}:{lastNodeHash} with 1-hour TTL. If the same range is needed again, the cached summary is reused.
- **Fallback:** If summarization itself fails, truncate oldest messages.

10.3 Budget API

GET /api/branches/:id/context-budget returns:

```
{ "used": 45230, "available": 82770, "total": 128000, "percentage": 35.3,
"needsSummarization": false, "recentMessages": 15, "summarizedMessages": 0 }
```

11. Module 9: Export Engine

Owners: Dev B (API) + Dev A (UI)

11.1 JSON Export

Returns the raw tree structure with all nodes, branches, and metadata. Suitable for backup, re-import, or programmatic consumption.

11.2 Markdown Export

Walks the ancestor chain of the selected branch and formats each message as:

```
## User\n{content}\n\n## Assistant (gpt-4o)\n{content}\n\n
```

Branch annotations are added as Markdown comments indicating fork points. The user can optionally prune dead-end branches before export.

11.3 Snapshot / Checkpoint Export

POST /api/snapshots creates an immutable checkpoint: snapshotHash = SHA-256 of ordered content hashes in the branch. GET /api/snapshots/:hash retrieves the exact state. This enables deterministic state restoration.

12. Module 10: Launch Preparation

Owner: Dev D (with support from all)

12.1 Landing Page

Deployed on Vercel (separate from the main app). Sections: hero with demo GIF, problem/solution narrative, 3 feature cards with screenshots, early-access CTA. Built with Tailwind CSS. Dev D owns this and can work on it in parallel with all other modules.

12.2 Security Hardening Checklist

- All endpoints require Clerk authentication
- Workspace ownership verified on every request
- Input sanitization on all user-provided text
- CORS allows only the production domain
- Rate limiting: 20 LLM requests/min per user
- Message size limit: 50,000 characters
- All secrets in SSM Parameter Store, zero hardcoded keys
- Clerk production keys (not dev keys) in production

12.3 Onboarding

Dev C creates a guided onboarding experience: new users get a pre-created workspace with a sample conversation tree demonstrating branching. Includes one pre-made branch comparing GPT-4o vs Claude on the same prompt.

12.4 Load Testing

Dev D runs Artillery or k6 scripts simulating 50 concurrent users creating branches and streaming responses. Target: backend handles 50 concurrent users, React Flow remains smooth at 100+ nodes, all queries < 100ms at 500 rows.

12.5 Analytics

PostHog or Plausible for landing page and app. Key events tracked: sign_up, first_message, first_branch, model_switch, export.

13. Parallel Work Matrix

This matrix shows what each developer can work on simultaneously. All modules can start from Day 1 because of contract-first development and mock-based decoupling.

Phase	Dev A (Frontend)	Dev B (Backend)	Dev C (LLM/Full-stack)	Dev D (DevOps/Product)
Phase 1: Foundation	React app scaffold, React Flow canvas, Clerk auth UI, chat input component, message node component	Fastify setup, DB schema + migrations, workspace/message CRUD APIs, publish API contracts	LLM adapter with model registry, streamText() integration, Clerk backend middleware, test streaming locally	AWS account + IAM, ECS deployment, RDS + Redis provisioning, CI/CD pipeline, Sentry setup
Phase 2: Branching	Context menu on nodes, branch creation modal, dagre re-layout with branch paths, active/inactive branch styling, branch sidebar, dual-view (canvas + linear chat)	Branch fork API, branch switching, full tree endpoint, branch CRUD, ancestor chain query optimization, Redis caching for tree	Branch-aware context assembly, branch isolation verification, content-addressable hashing, token counting utility, SSE streaming endpoint	QA of branch isolation, performance baseline (10 branches / 50 msgs each), staging vs prod separation
Phase 3: Multi-model + Budget	Model selector dropdown (per-branch), model badge on message nodes, context budget progress bar, export modal, UI polish (loading skeletons, error states, toasts)	Model field on messages, branch model endpoint, context budget API, export API (JSON + Markdown), API hardening, DB query optimization	Anthropic Claude adapter, progressive summarization, context budget algorithm, Redis summary caching, LLM error handling (retry/fallback), merge concat strategy	Multi-model QA, load testing, landing page build, demo video recording, legal docs, production environment
Phase 4: Stabilize + Launch	Accessibility pass, responsive layout, analytics integration, landing page final polish, Open Graph meta tags	Security hardening (ownership checks, input sanitization, CORS), connection pool tuning, migration idempotency	Onboarding flow with seed data, content moderation (input validation), usage tracking endpoint	Production deploy, load test, smoke test, launch post drafts, war room protocol, public launch coordination

14. Risk Register

Risk	Likelihood	Impact	Mitigation
React Flow + streaming performance	High	High	Buffer tokens with requestAnimationFrame batching. Only re-render active streaming node via Zustand selective subscriptions. Test with 50+ nodes early.
LLM API rate limits or outages	Medium	High	Retry with exponential backoff. Fallback model if primary fails. Cache responses by content hash. User-facing rate limits lower than API limits.
Dagre layout breaks with complex branches	Medium	Medium	Test with 5+ branches from single node early. Fallback: switch to ELK.js. Limit viewport to 200 nodes with pagination.
Context assembly incorrectness	Medium	Critical	Integration tests for ancestor chain queries. Test edge cases: fork from root, fork from fork, 100+ message branches. Dev D manually verifies.
Scope creep	High	High	Dev D is Chief Scope Reducer. Every new idea goes to v2 backlog. Daily standup includes explicit scope check.
Developer illness	Low	High	No single point of failure. Cross-train: Dev C can cover backend, Dev A can do basic API work. Document all decisions within 1 hour.

15. Non-Negotiable Architectural Decisions

These three decisions compound over the entire build and must not be compromised:

15.1 Content-Addressable Hashing on Nodes

Every message gets a SHA-256 hash of its role + content + parent_id. This costs minimal implementation effort but enables future checkpoint/restore, deduplication, and deterministic snapshots without any schema redesign.

15.2 Vercel AI SDK Abstraction

The AI SDK normalizes OpenAI, Anthropic, and future providers behind a unified interface. Adding a new model is a one-line registry entry. Never call provider APIs directly; always go through the SDK layer.

15.3 Adjacency List + ltree in PostgreSQL

Start with parent_id adjacency list and recursive CTEs. Enable the ltree extension from Day 1. If ancestor queries become slow at scale, add a materialized ltree path column as a performance optimization without changing any application logic. This approach scales to thousands of branches.

16. Feature Scope: What Ships vs. What Doesn’t

16.1 Must Ship

- User authentication (Clerk)
- Single-model conversation (GPT-4o) with streaming
- Branch creation from any message node
- Conversation tree visualization (React Flow + dagre)
- Basic context budget management
- Multi-model support (add Claude)
- Export (JSON + Markdown)
- Landing page

16.2 Defer to v2

- Highlight-and-branch (complex coordinate transforms, 4–5 days of frontend work)
- Advanced merge strategies (summarize/diff — needs user feedback first)
- Real-time collaboration (entire separate product surface — requires CRDT/Yjs integration)
- Drag-and-drop message reorganization (complex topology validation)
- Team workspaces and RBAC
- Desktop app (Electron/Tauri)